MODULE 28 · EXAM STRATEGY - ALL BLOCKS

Practice paper 4: final rehearsal

Thirty new questions under exam conditions: no paper, about 80 seconds each, in the exam's proportions.

QUESTIONS 30 TIMING about 80 s each MODULE 28 of 28

admin@sthithapragna.com

Exam conditions

- **No paper, no notes.** Exactly as on the day.
- About **80 seconds** per question. Pause only to keep to that pace.
- Commit to an answer before each walkthrough, even when unsure.
- Score it, then compare with papers 1 to 3.

What is the output of this program?

```
1  print(-3 ** 2, (-3) ** 2, 2 ** -1)
```

A 9 9 0.5

B -9 9 0.5

C -9 9 -2

D -9 -9 0.5

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(-3 ** 2, (-3) ** 2, 2 ** -1)
```

A `9 9 0.5`
*** binds tighter than the minus on its left: -(3 ** 2).*

B `-9 9 0.5`
-(3 ** 2); then (-3) squared; then 1 / 2 as a float.

C `-9 9 -2`
*2 ** -1 is one half, not -2.*

D `-9 -9 0.5`
The brackets make (-3) the base, so it is squared to 9.

BLOCK 1

A minus on the left of ** is applied after the power.

What is the output of this program?

```
for ch in "hello":
    if ch == "l":
        break
    print(ch, end="")
```

A he

B hel

C heo

D hello

◆ PAUSE AND DECIDE

What is the output of this program?

```
for ch in "hello":
    if ch == "l":
        break
    print(ch, end="")
```

A **he**

h and e print; the first l triggers break.

B **hel**

break comes before the print, so l is not printed.

C **heo**

break ends the loop; it does not skip to the next character.

D **hello**

The loop stops at the first l.

BLOCK 2

break ends the whole loop; continue would skip one pass.

What is the output of this program?

```
1  data = [1, [1, 2], 2]
2  print(data.count(1), 2 in data, len(data[1]))
```

A 2 True 2

B 1 True 2

C 1 False 2

D 2 True 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
data = [1, [1, 2], 2]
print(data.count(1), 2 in data, len(data[1]))
```

A `2 True 2`

count looks only at top-level elements: the 1 inside the list does not count.

B `1 True 2`

One top-level 1; 2 is a top-level element; the inner list has 2.

C `1 False 2`

The last element is 2, so 2 in data is True.

D `2 True 3`

len(data[1]) is the inner list's length.

BLOCK 3

count, in and len look at top-level elements only.

What is the output of this program?

```
def f(x=None):
    return x is None

print(f(), f(0))
```

A True False

B True True

C False False

D None 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def f(x=None):
2      return x is None
3
4  print(f(), f(0))
```

A `True False`

f() uses the default None; 0 is not None.

B `True True`

0 is falsy, but it is not None.

C `False False`

With no argument, x is None, so is gives True.

D `None 0`

The function returns the result of the is test.

BLOCK 4

is None tests for None itself, not for any falsy value.

What is the output of this program?

```
1  s = "a#b"  # a comment
2  print(len(s))
```

A 1

B The code raises a SyntaxError

C 5

D 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  s = "a#b"  # a comment
2  print(len(s))
```

A `1`

A # inside quotes is just a character.

B `The code raises a SyntaxError`

Comments may follow code on the same line.

C `5`

The comment after the code is not part of the string.

D `3`

a, # and b; the real comment is ignored.

BLOCK 1

inside quotes is text. # outside quotes starts a comment.

What is the output of this program?

```
n = 7
while n > 0:
    n -= 2
else:
    print(n)
```

A 1

B -1

C Nothing is printed

D 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 7
2  while n > 0:
3      n -= 2
4  else:
5      print(n)
```

A `1`

The loop runs once more: 1 > 0, so n becomes -1.

B `-1`

n: 5, 3, 1, -1. Then the test fails and the else prints.

C `Nothing is printed`

No break, so the loop's else runs.

D `0`

Odd minus 2 stays odd: n never equals 0.

BLOCK 2

The else sees the value that failed the test.

What is the output of this program?

```
1  text = "  Hi There "
2  print(text.strip().split()[1].upper())
```

A THERE

B HI

C There

D The code raises an IndexError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  text = "  Hi There "
2  print(text.strip().split()[1].upper())
```

A THERE

strip, split on spaces, take index 1, then upper.

B HI

Index 1 is the second word.

C There

upper() capitalises every letter.

D The code raises an IndexError

split gives two words, so index 1 exists.

BLOCK 3

Chained methods apply left to right.

What is the output of this program?

```
def total(nums):
    if not nums:
        return 0
    return nums[0] + total(nums[1:])

print(total([2, 4, 6]))
```

A 2

B 246

C 12

D The code raises a RecursionError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def total(nums):
2      if not nums:
3          return 0
4      return nums[0] + total(nums[1:])
5
6  print(total([2, 4, 6]))
```

A 2

The recursion adds every element.

B 246

The values are numbers, so + adds them.

C 12

2 + (4 + (6 + 0)).

D `The code raises a RecursionError`

Each call has a shorter list, reaching the empty list.

BLOCK 4

Each recursive call must move closer to the base case.

What is the output of this program?

```
1  x, y = 1, 2
2  x, y = y, x + y
3  print(x, y)
```

A 2 3

B 2 4

C 1 3

D 3 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x, y = 1, 2
2  x, y = y, x + y
3  print(x, y)
```

A 2 3

The right side uses the old x and y: 2, then 1 + 2.

B 2 4

x + y uses the old x, 1, not the new one.

C 1 3

x is given the old y, 2.

D 3 2

The pair is assigned in order: x first, then y.

BLOCK 1

The right side is evaluated completely before any name changes.

What is the output of this program?

```
for i in range(1, 4):
    if i == 2:
        continue
    for j in range(i):
        print(i, end="")
```

A 122333

B 13

C 1223

D 1333

◆ PAUSE AND DECIDE

What is the output of this program?

```
for i in range(1, 4):
    if i == 2:
        continue
    for j in range(i):
        print(i, end="")
```

A 122333

continue skips the whole of the i = 2 pass.

B 13

The inner loop prints i, i times.

C 1223

When i is 3, the inner loop prints 3 three times.

D **1333**

i = 1 prints 1; i = 2 is skipped; i = 3 prints 333.

BLOCK 2

continue in the outer loop skips the inner loop for that pass.

What is the output of this program?

```
prices = {"b": 2, "a": 1}
prices["c"] = 3
del prices["b"]
print(list(prices), sum(prices.values()))
```

A ['b', 'a', 'c'] 6

B ['a', 'b', 'c'] 4

C [1, 3] 4

D ['a', 'c'] 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  prices = {"b": 2, "a": 1}
2  prices["c"] = 3
3  del prices["b"]
4  print(list(prices), sum(prices.values()))
```

A `['b', 'a', 'c'] 6`
del removed b.

B `['a', 'b', 'c'] 4`
b was deleted, and keys are not sorted.

C `[1, 3] 4`
list(prices) gives the keys.

D `['a', 'c'] 4`
b is deleted; keys stay in insertion order; 1 + 3.

BLOCK 3

list(d) gives keys, in insertion order.

What is the output of this program?

```
def safe_div(a, b):
    try:
        r = a / b
    except ZeroDivisionError:
        return "inf"
    else:
        return round(r, 1)

print(safe_div(1, 3), safe_div(1, 0))
```

A 0.3 0.0

B 0.3 inf

C 0.33 inf

D The code raises a ZeroDivisionError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def safe_div(a, b):
2      try:
3          r = a / b
4      except ZeroDivisionError:
5          return "inf"
6      else:
7          return round(r, 1)
8
9  print(safe_div(1, 3), safe_div(1, 0))
```

A `0.3 0.0`
The except branch returns the string inf.

B `0.3 inf`
1 / 3 rounds to 0.3 in else; 1 / 0 is handled.

C `0.33 inf`
round(r, 1) keeps one decimal place.

D `The code raises a ZeroDivisionError`
It is caught inside safe_div.

BLOCK 4

else runs only when the try block succeeded.

What is the output of this program?

```
1  print(int(7.9) + int("7") * 2)
```

A 22

B The code raises a ValueError

C 21

D 28

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(int(7.9) + int("7") * 2)
```

A 22
int(7.9) cuts to 7; it does not round to 8.

B The code raises a ValueError
int("7") is a valid conversion.

C 21
7 + 7 * 2: int() truncates, then * before +.

D 28
** comes before +: 7 + 14.*

BLOCK 1

int() of a float truncates. Then precedence as usual.

What is the output of this program?

```
items = []
if items:
    print("full")
elif items == []:
    print("empty")
else:
    print("?")
```

A `full`

B `?`

C `empty`

D Nothing is printed

◆ PAUSE AND DECIDE

What is the output of this program?

```
items = []
if items:
    print("full")
elif items == []:
    print("empty")
else:
    print("?")
```

A `full`
An empty list is falsy.

B ?
The elif test is True.

C **empty**
if [] is False; [] == [] is True.

D Nothing is printed
One branch of the chain matches.

BLOCK 2

Empty collections are falsy.

What is the output of this program?

```
1  print([x * y for x in range(1, 3) for y in range(2)])
```

A [0, 1, 2]

B [0, 2]

C [0, 1, 0, 2]

D [1, 2, 2, 4]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print([x * y for x in range(1, 3) for y in range(2)])
```

A `[0, 1, 2]`
Two loops: every x is paired with every y.

B `[0, 2]`
y takes both 0 and 1 for each x.

C **`[0, 1, 0, 2]`**
x = 1: 0, 1. x = 2: 0, 2.

D `[1, 2, 2, 4]`
range(2) gives y = 0 and 1, not 1 and 2.

BLOCK 3

Two for clauses in a comprehension work like nested loops.

What happens when this program runs?

```
try:
    print(1 / 0)
except ZeroDivisionError:
    print(undefined)
print("end")
```

A end is printed

B The ZeroDivisionError is handled, and end is printed

C Nothing is printed; the program raises a SyntaxError

D The program stops with a NameError; end is not printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  try:
2      print(1 / 0)
3  except ZeroDivisionError:
4      print(undefined)
5  print("end")
```

A end is printed
The except branch itself fails.

B The ZeroDivisionError is handled, and end is printed
Handling starts, but the branch raises a new error.

C Nothing is printed; the program raises a SyntaxError
An undefined name is a runtime error.

D The program stops with a NameError; end is not printed
The error inside except is new, and nothing handles it.

BLOCK 4

An exception raised inside an except branch is a new, unhandled problem.

What is the output of this program?

```
1  print("ab" * 2 < "abc", len("ab" * 0))
```

A False 0

B True 0

C True 2

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print("ab" * 2 < "abc", len("ab" * 0))
```

A `False 0`
Compare character by character: a=a, b=b, then a < c.

B `True 0`
"abab" < "abc" at the third character; "ab" * 0 is empty.

C `True 2`
Repeating zero times gives an empty string.

D `The code raises a TypeError`
*str * int is repetition.*

BLOCK 1

Strings compare at the first differing character, not by length.

What happens when this program runs?

```
for k in range(0):
    pass
print(k)
```

A -1 is printed

B The program raises a NameError

C 0 is printed

D Nothing is printed; the program raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
for k in range(0):
    pass
print(k)
```

A -1 is printed
range(0) produces nothing.

B The program raises a NameError
Zero passes: k is never assigned.

C 0 is printed
range(0) has no values, not a 0.

D Nothing is printed; the program raises a SyntaxError
The grammar is fine.

BLOCK 2

Paired with paper 1, question 22: zero passes means the loop variable never exists.

What is the output of this program?

```
1  t = (5)
2  u = (5,)
3  print(t * 2, u * 2)
```

A (5, 5) (5, 5)

B 10 (5, 5)

C 10 10

D (10,) (5, 5)

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  t = (5)
2  u = (5,)
3  print(t * 2, u * 2)
```

A `(5, 5) (5, 5)`
(5) has no comma, so t is the int 5.

B **`10 (5, 5)`**
t is 5, so t * 2 is 10; u is a tuple, repeated.

C `10 10`
*(5,) is a tuple, so * repeats it.*

D `(10,) (5, 5)`
t is not a tuple at all.

BLOCK 3

The comma makes a tuple.

What is the output of this program?

```
hits = 0
def hit():
    global hits
    hits += 1
    return hits

for _ in range(3):
    hit()
print(hits, hit())
```

A 3 4

B 3 3

C 0 1

D 4 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  hits = 0
2  def hit():
3      global hits
4      hits += 1
5      return hits
6
7  for _ in range(3):
8      hit()
9  print(hits, hit())
```

A `3 4`

Three calls make hits 3; the fourth call, in print, makes it 4.

B `3 3`

hit() inside print runs again.

C `0 1`

global hits makes the function change the outer counter.

D `4 4`

print evaluates hits before calling hit().

BLOCK 4

Arguments are evaluated left to right.

What is the output of this program?

```
1  print(7.5 // 2, 7.5 % 2)
```

A 3 1.5

B 3.75 1.5

C 3.0 1

D 3.0 1.5

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(7.5 // 2, 7.5 % 2)
```

A `3 1.5`
A float operand makes // return a float: 3.0.

B `3.75 1.5`
// floors; it does not divide exactly.

C `3.0 1`
% on a float gives a float: 1.5.

D **`3.0 1.5`**
7.5 // 2 is 3.0; 3.0 * 2 + 1.5 is 7.5.

BLOCK 1

// and % on floats give floats.

What is the output of this program?

```
1  for i in range(3):
2      print("*" * (i + 1), end=" ")
```

A `* ** ***`

B `*** ** *`

C `** *** ****`

D `* * *`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  for i in range(3):
2      print("*" * (i + 1), end=" ")
```

A `* ** ***`

Pass i prints i + 1 stars, then a space.

B `*** ** *`

The count grows with i.

C `** *** ****`

range(3) starts at 0, so the first line has 1 star.

D `* * *`

"" * (i + 1) repeats the star.*

BLOCK 2

range(3) starts at 0. String * int repeats.

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a[::-1]
3  b[0] = 9
4  print(a, b)
```

A [9, 2, 3] [9, 2, 1]

B [1, 2, 3] [9, 2, 3]

C [3, 2, 1] [9, 2, 1]

D [1, 2, 3] [9, 2, 1]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a[::-1]
3  b[0] = 9
4  print(a, b)
```

A `[9, 2, 3] [9, 2, 1]`
A slice is a copy; changing b cannot reach a.

B `[1, 2, 3] [9, 2, 3]`
[::-1] reverses: b starts as [3, 2, 1].

C `[3, 2, 1] [9, 2, 1]`
Slicing never changes the original list.

D **`[1, 2, 3] [9, 2, 1]`**
b is a reversed copy; a is untouched.

BLOCK 3

Paired with paper 2, question 7: a slice makes a copy; += keeps one shared list.

What is the output of this program?

```
def nums():
    n = 0
    while True:
        yield n
        n += 1

for x in nums():
    if x > 2:
        break
    print(x, end="")
```

A 0123

B 012

C The loop never ends

D 123

◆ PAUSE AND DECIDE

What is the output of this program?

```
def nums():
    n = 0
    while True:
        yield n
        n += 1

for x in nums():
    if x > 2:
        break
    print(x, end="")
```

A `0123`

The break happens before 3 is printed.

B **`012`**

The generator is infinite; break stops the loop at 3.

C `The loop never ends`

break ends the loop once x is 3.

D `123`

n starts at 0, and 0 is yielded first.

BLOCK 4

A generator produces values only when asked, so an infinite one is safe with break.

> Each expression is evaluated on its own.

Which two expressions raise a TypeError? (Choose two.)

A `"3" * "3"`

B `"3" * 3`

C `len(42)`

D `int("3")`

◆ PAUSE AND DECIDE

Which two expressions raise a TypeError? (Choose two.)

A `"3" * "3"`

You cannot multiply a str by a str.

B `"3" * 3`

*str * int repeats: "333".*

C `len(42)`

An int has no length.

D `int("3")`

A valid conversion: 3.

BLOCK 1

Wrong type for the operation: TypeError.

What is the output of this program?

```
x = 1
while x < 100:
    x = x * 2 + 1
print(x)
```

A 63

B 100

C 64

D 127

◆ PAUSE AND DECIDE

What is the output of this program?

```
x = 1
while x < 100:
    x = x * 2 + 1
print(x)
```

A 63

63 < 100, so the loop runs once more.

B 100

x never equals 100; it jumps from 63 to 127.

C 64

*The update is x * 2 + 1, not x * 2.*

D 127

1, 3, 7, 15, 31, 63, 127: the first value not below 100.

BLOCK 2

A while loop leaves with the first value that fails the test.

> Each line is run on its own.

Which two lines print True? (Choose two.)

A `print("Abc".isupper())`

B `print("abc".isalpha())`

C `print("123".isdigit())`

D `print("a b".isalnum())`

◆ PAUSE AND DECIDE

Which two lines print True? (Choose two.)

A `print("Abc".isupper())`
b and c are lower case.

B `print("abc".isalpha())`
Letters only.

C `print("123".isdigit())`
Digits only.

D `print("a b".isalnum())`
The space is not a letter or a digit.

BLOCK 3

The is methods test every character.

What is the output of this program?

```
1  def add(d, k):
2      d[k] = d.get(k, 0) + 1
3
4  inventory = {}
5  add(inventory, "a")
6  add(inventory, "a")
7  print(inventory)
```

A {}

B {'a': 1}

C The code raises a KeyError

D {'a': 2}

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def add(d, k):
2      d[k] = d.get(k, 0) + 1
3
4  inventory = {}
5  add(inventory, "a")
6  add(inventory, "a")
7  print(inventory)
```

A `{}`
The function changes the dictionary it is given.

B `{'a': 1}`
The second call adds one more.

C `The code raises a KeyError`
get supplies 0 the first time.

D `{'a': 2}`
The same dictionary is updated twice through d.

BLOCK 4

A function can change a dictionary or list it is given.

What is the output of this program?

```
for n in range(2, 5):
    if 7 % n == 0:
        break
else:
    print("prime")
```

A Nothing is printed

B prime, three times

C prime

D The code raises a ZeroDivisionError

◆ PAUSE AND DECIDE

What is the output of this program?

```
for n in range(2, 5):
    if 7 % n == 0:
        break
else:
    print("prime")
```

A `Nothing is printed`
No divisor is found, so no break fires: the else runs.

B `prime, three times`
The else runs once, after the loop.

C **`prime`**
7 % 2, 7 % 3 and 7 % 4 are all non-zero: no break.

D `The code raises a ZeroDivisionError`
range(2, 5) never gives 0.

BLOCK 2

A search with for-else: the else means nothing was found.

> Consider Python exceptions in general.

Which two statements are true? (Choose two.)

A except ArithmeticError: catches a ZeroDivisionError

B IndexError is a subclass of KeyError

C An exception not handled in a function passes to its caller

D assert raises a ValueError when its condition is false

◆ PAUSE AND DECIDE

Which two statements are true? (Choose two.)

A except ArithmeticError: catches a ZeroDivisionError

ZeroDivisionError is a kind of ArithmeticError.

B IndexError is a subclass of KeyError

They are siblings under LookupError.

C An exception not handled in a function passes to its caller

It travels up until something handles it.

D assert raises a ValueError when its condition is false

assert raises AssertionError.

BLOCK 4

Parents catch children. Unhandled exceptions travel to the caller.

Paper 4, by block

BLOCK	QUESTIONS IN THIS PAPER	MODULES TO REVISIT
1: Fundamentals	1, 5, 9, 13, 17, 21, 25	1 to 5
2: Control flow	2, 6, 10, 14, 18, 22, 26, 29	6 to 10
3: Data collections	3, 7, 11, 15, 19, 23, 27	11 to 15
4: Functions and exceptions	4, 8, 12, 16, 20, 24, 28, 30	16 to 21

Reading the final rehearsal

PAPER 4 SCORE	WHAT IT MEANS	WHAT TO DO
27 or more	ready, with a margin	sit the exam while it is fresh
24 to 26	ready, with a small margin	revisit the block where misses fell; then sit it
21 to 23	at the pass line, with no margin	rework your weakest block and modules 22 and 23 first
Below 21	not ready yet	rework the weakest block, then retake papers 1 to 4

BEFORE YOU BOOK

Aim to sit the exam with a margin above 70%, not on the line. Retakes are allowed after 7 days, but passing first time is the goal.